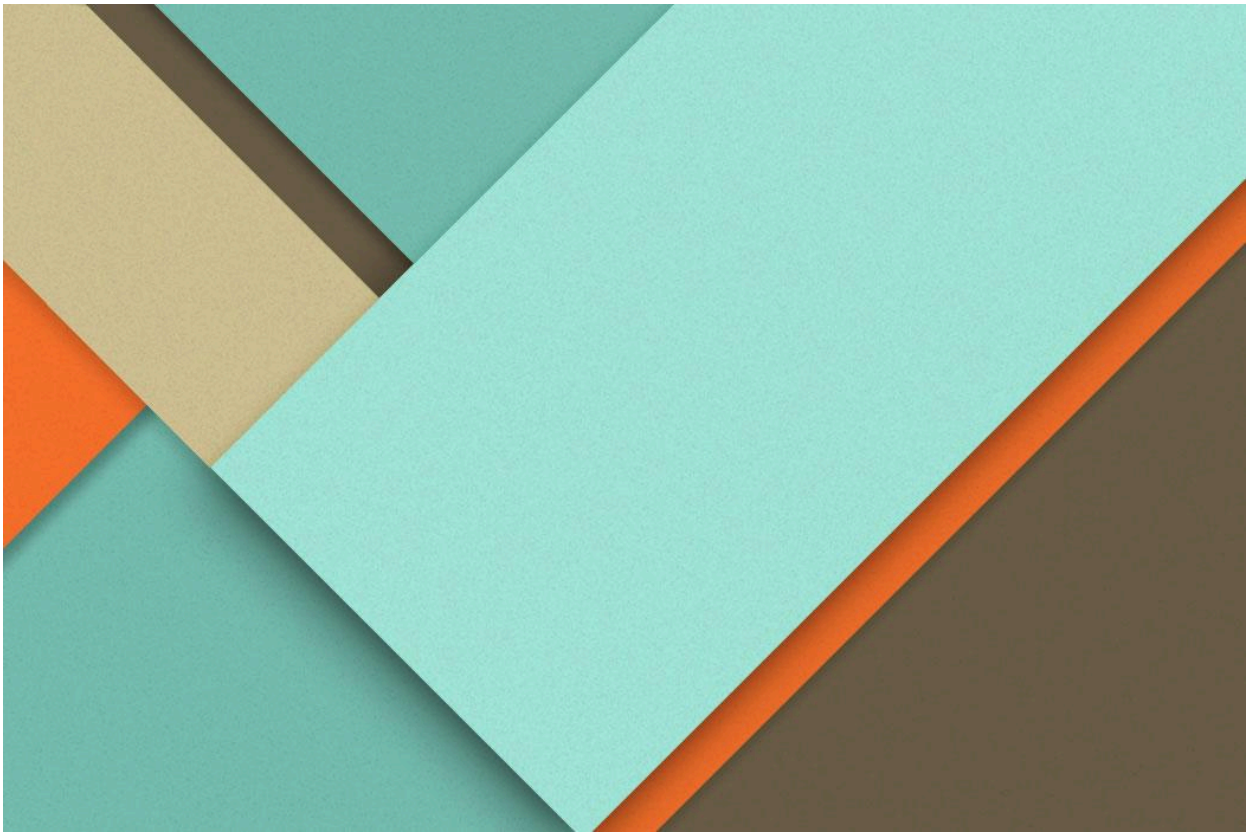


Gen-Ai Hands-On-2

20.03.2026

ABHISHEK S NAIR
SRN: PES2UG23CS022
SEM: 6
SECTION: A

Notebook-1 [LangChain]

LangChain provides a unified interface for working with multiple language models, allowing seamless switching between providers with minimal code modifications.

Language models process input as tokens rather than words, and both cost and memory usage depend on token consumption. Therefore, understanding tokens is essential, as they directly impact pricing and context window limitations.

Temperature

7. Experiment: Consistency vs. Creativity

We will ask both models to "Define the word 'Idea' in one sentence." We will run the code **TWICE** for each model.

Hypothesis:

- The Focused model (Temp=0) should say the *exact same thing* both times.
- The Creative model (Temp=1) should say *different things*.

```
prompt = "Define the word 'Idea' in one sentence."

print("--- FOCUSED (Temp=0) ---")
print(f"Run 1: {llm_focused.invoke(prompt).content}")
print(f"Run 2: {llm_focused.invoke(prompt).content}")
```

[4] ✓ 49.9s Python

```
--- FOCUSED (Temp=0) ---
Run 1: An idea is a thought, concept, or mental impression that originates in the mind.
Run 2: An idea is a thought, concept, or mental impression that exists in the mind.
```

```
print("--- CREATIVE (Temp=1) ---")
print(f"Run 1: {llm_creative.invoke(prompt).content}")
print(f"Run 2: {llm_creative.invoke(prompt).content}")
```

[5] ✓ 12.5s Python

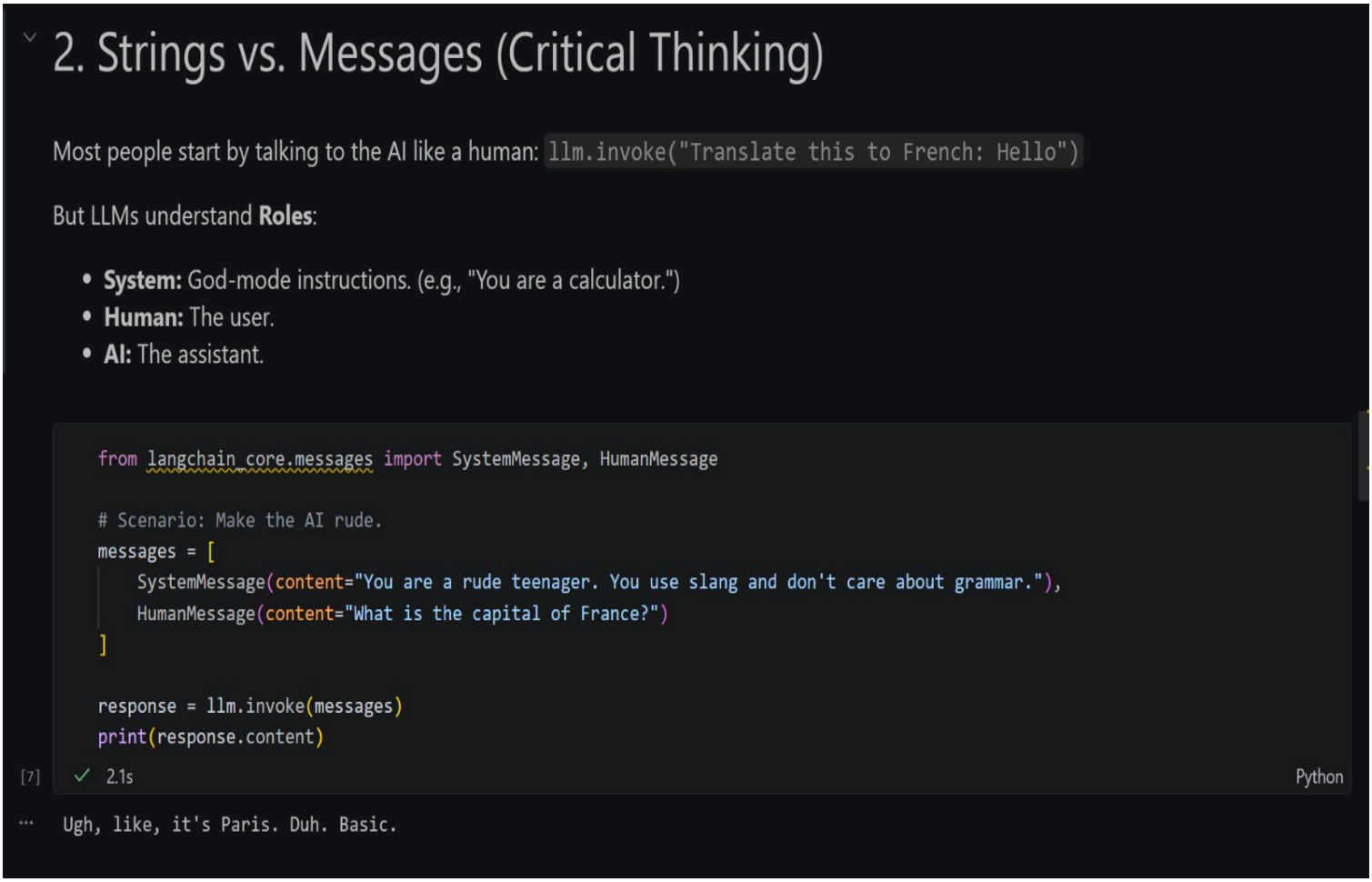
```
--- CREATIVE (Temp=1) ---
Run 1: An idea is a concept, thought, or mental impression that forms the basis for understanding, action, or creation.
Run 2: An idea is a thought, concept, or mental impression that originates in the mind, often serving as a basis for understanding,
```

Observation

This experiment demonstrates that the temperature parameter controls the randomness and creativity of model outputs.

- *At temperature = 0, the outputs remain consistent and deterministic across multiple runs.*
- *As the temperature increases (e.g., 1 or 2), responses become more diverse and expressive.*
- *Higher temperature values produce more creative outputs but reduce predictability.*

Strings vs Messages



Observation

This example highlights that LLM responses vary depending on the role of the message (System vs Human).

- *The SystemMessage defines the model's behavior, tone, and personality.*
- *Changing the system role (e.g., making the model "rude") significantly alters the response, even for simple queries.*
- *This demonstrates the strong influence of system-level instructions on output generation.*

Prompt Templates

Observation

This example shows how structured prompts improve interaction design.

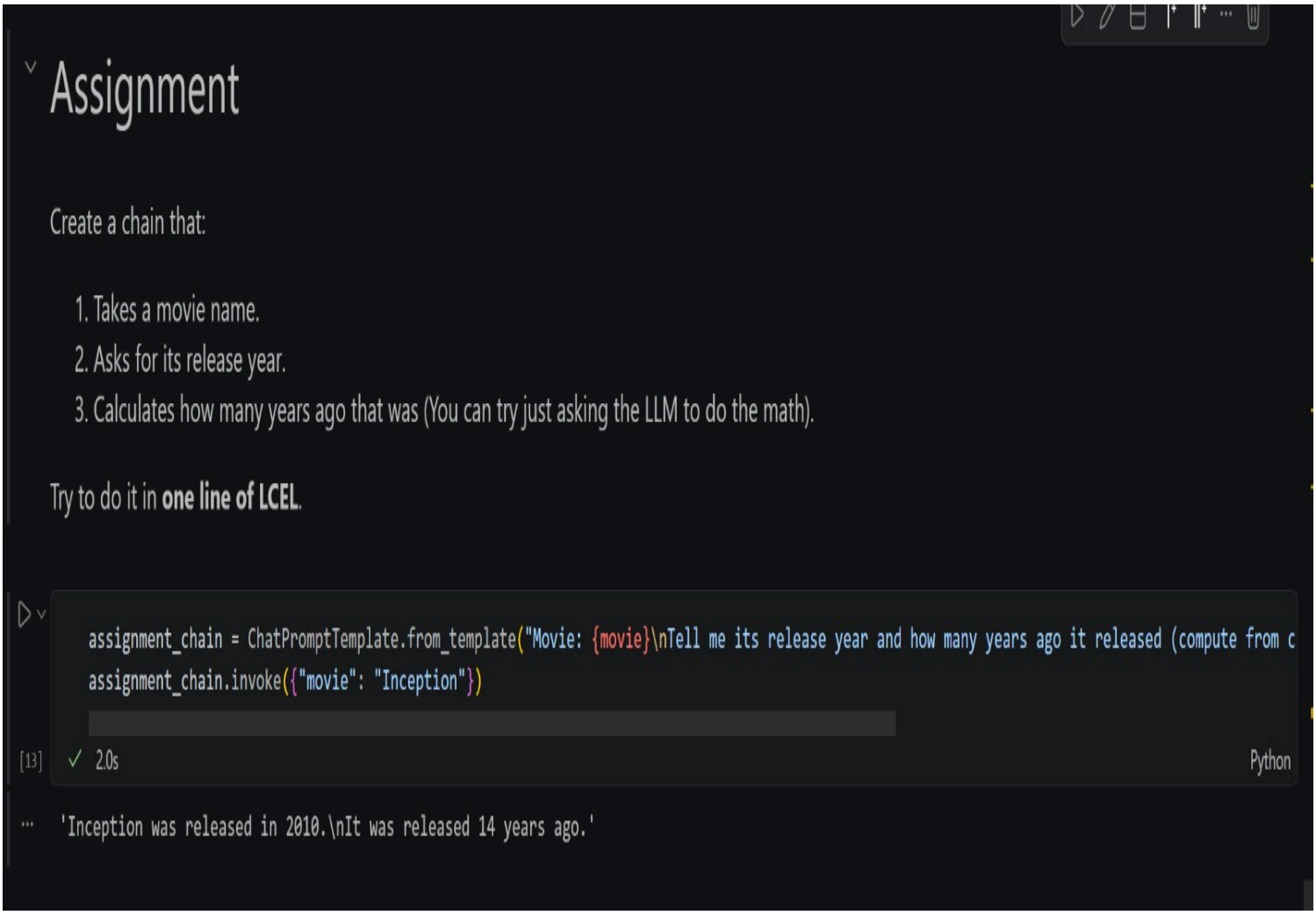
- **The use of system and human roles organizes the conversation effectively.**
- **Printing template.input_variables helps verify required inputs before execution.**
- **This reduces errors by ensuring all necessary parameters are provided.**

LCEL (LangChain Expression Language)

Observation

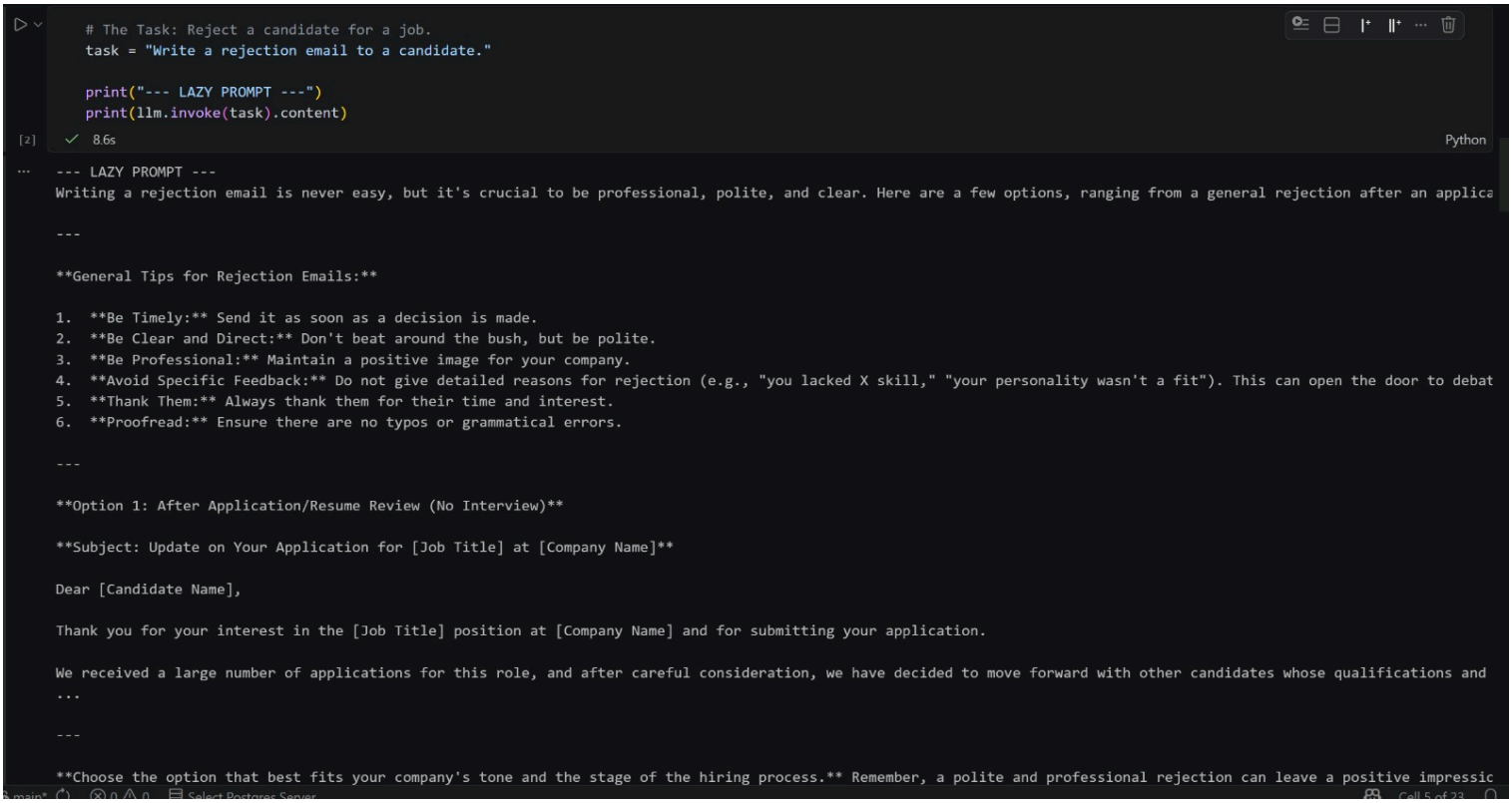
This example demonstrates how LCEL enables the creation of modular and readable pipelines using the pipe (|) operator.

- *The output of one component is directly passed to the next.*
- *This improves clarity and reduces complexity.*
- *It enables easy modification, such as swapping models or inserting intermediate steps (e.g., spell-checking).*
- *Overall, LCEL enhances scalability, maintainability, and structure in AI workflows.*



Notebook-2

Co-STAR Framework



```
# The Task: Reject a candidate for a job.
task = "Write a rejection email to a candidate."

print("--- LAZY PROMPT ---")
print(llm.invoke(task).content)
```

[2] ✓ 8.6s Python

```
--- LAZY PROMPT ---
Writing a rejection email is never easy, but it's crucial to be professional, polite, and clear. Here are a few options, ranging from a general rejection after an applica

---

**General Tips for Rejection Emails:**

1.  **Be Timely:** Send it as soon as a decision is made.
2.  **Be Clear and Direct:** Don't beat around the bush, but be polite.
3.  **Be Professional:** Maintain a positive image for your company.
4.  **Avoid Specific Feedback:** Do not give detailed reasons for rejection (e.g., "you lacked X skill," "your personality wasn't a fit"). This can open the door to debat
5.  **Thank Them:** Always thank them for their time and interest.
6.  **Proofread:** Ensure there are no typos or grammatical errors.

---

**Option 1: After Application/Resume Review (No Interview)**

**Subject: Update on Your Application for [Job Title] at [Company Name]**

Dear [Candidate Name],

Thank you for your interest in the [Job Title] position at [Company Name] and for submitting your application.

We received a large number of applications for this role, and after careful consideration, we have decided to move forward with other candidates whose qualifications and
...

---

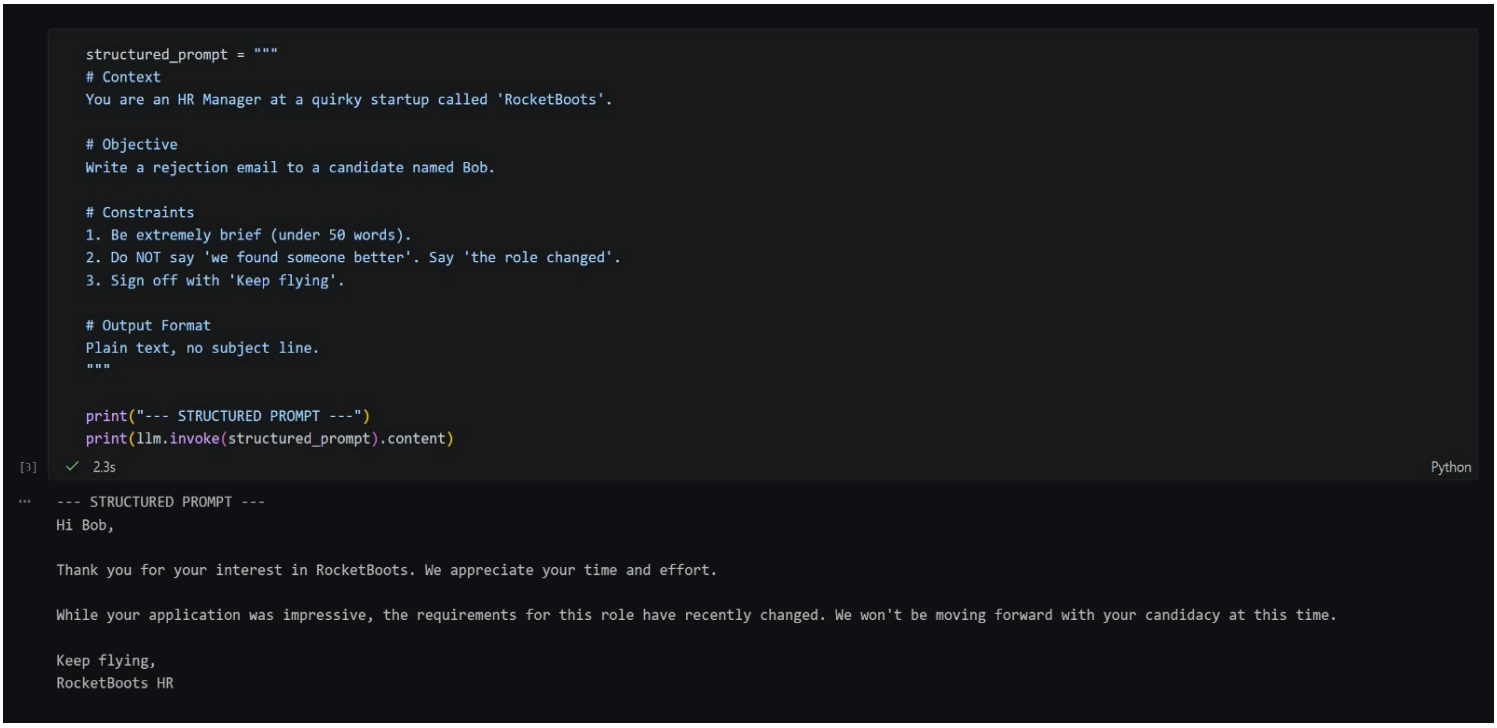
**Choose the option that best fits your company's tone and the stage of the hiring process.** Remember, a polite and professional rejection can leave a positive impressic
```

Observation

This comparison illustrates the importance of structured prompting.

- *A poorly designed (lazy) prompt produces generic and overly verbose responses.*
- *The CO-STAR framework introduces structure through context, objective, tone, audience, and format.*
- *This leads to more precise, relevant, and controlled outputs.*
- *Hence, better prompt design directly improves response quality.*

Hallucination vs Creativity



```
structured_prompt = """
# Context
You are an HR Manager at a quirky startup called 'RocketBoots'.

# Objective
Write a rejection email to a candidate named Bob.

# Constraints
1. Be extremely brief (under 50 words).
2. Do NOT say 'we found someone better'. Say 'the role changed'.
3. Sign off with 'Keep flying'.

# Output Format
Plain text, no subject line.
"""

print("--- STRUCTURED PROMPT ---")
print(llm.invoke(structured_prompt).content)
```

[3] ✓ 2.3s Python

```
--- STRUCTURED PROMPT ---
Hi Bob,

Thank you for your interest in RocketBoots. We appreciate your time and effort.

While your application was impressive, the requirements for this role have recently changed. We won't be moving forward with your candidacy at this time.

Keep flying,
RocketBoots HR
```

Observation

This example shows how the model generates responses when information is incomplete.

- *The model fills missing details by predicting the most probable explanation.*
- *In this case, it assumed a common reason (“role changed”) due to lack of explicit input.*
- *This behavior represents creative inference rather than true hallucination.*
- *It highlights the importance of providing clear and complete input for accurate outputs.*

Assignment

Write a structured prompt to generate a **Python Function**.

- **Context:** You are a Senior Python Dev.
- **Objective:** Write a function to reverse a string.
- **Constraint:** It must use recursion (no slicing `[::-1]`).
- **Style:** Include detailed docstrings.

```
assignment_prompt = """
# Context
You are a Senior Python Developer writing production-quality, readable code.

# Objective
Write a Python function that reverses a string.

# Constraints
1. Use recursion only.
2. Do not use slicing (no [::-1]).
3. Handle empty strings correctly.

# Style
1. Include detailed docstrings with Args, Returns, and Examples.
2. Add concise inline comments for the recursive logic.
3. Return only valid Python code (no explanations outside code).
"""

print(llm.invoke(assignment_prompt).content)
```

[9] ✓ 6.5s Python

```
```python
def reverse_string(s: str) -> str:
    """
```
```

Zero-shot vs Few-shot Prompting

Observation

1. *Zero-shot prompting*
    - *Only task instructions are provided without examples.*
    - *Faster and more concise but may produce generic or inconsistent results.*
2. *Few-shot prompting*
    - *Includes example inputs and outputs.*
    - *Improves accuracy, structure, and alignment with expected results.*
    - *Increases token usage but enhances reliability.*

Advanced Prompting Techniques

Observation

Prompt Engineering vs Fine-Tuning

1. *Hard Prompts (Prompt Engineering)*
    - *Modify only input text*
    - *Fast, cost-effective, and suitable for prototyping*
2. *Soft Prompts (Fine-Tuning)*
    - *Modify model parameters*
    - *Require more data, time, and cost*

- *Provide better domain-specific performance*

3. *Prompt engineering is flexible and iterative, whereas fine-tuning is suited for long-term improvements.*

Dynamic Few-Shot Prompting

1. *Selects the most relevant examples using techniques like semantic search*
2. *Uses only top-matching examples within the context window*
3. *Improves efficiency, scalability, and response relevance*

Notebook-3

Chain of Thought (CoT)

1. *Encourages step-by-step reasoning before producing the final answer*
2. *Improves performance in logical, mathematical, and coding tasks*
3. *Simple to implement using prompts like "Let's think step by step"*
4. *Slightly increases token usage*

Tree of Thought (ToT)

1. *Explores multiple reasoning paths instead of a single sequence*
2. *Evaluates alternatives and selects the best outcome*
3. *Useful for planning and complex decision-making*
4. *Higher computational cost*

Graph of Thought (GoT)

1. *Allows interconnected reasoning across multiple paths*
2. *Enables merging and refining ideas*
3. *Suitable for research and creative tasks*
4. *Most computationally expensive approach*

Notebook-4

RAG (Retrieval-Augmented Generation)

1. *Addresses limitations like hallucination and knowledge cutoff*
2. *Retrieves external data before generating responses*
3. *Improves factual accuracy and grounding*

Vectors and Similarity

1. *Text is converted into high-dimensional vectors*
2. *Similar meanings result in similar vector representations*
3. *Cosine similarity measures closeness between vectors*
4. *Values closer to 1 indicate high similarity*

Naive RAG

1. *Follows Retrieval → Augmentation → Generation pipeline*
2. *Uses external knowledge sources to improve accuracy*
3. *FAISS enables efficient similarity search*
4. *LCEL connects components into a structured pipeline*

Indexing Algorithms

Observation

1. *Large-scale vector search introduces scalability challenges*
2. *FAISS optimizes indexing and retrieval*
3. *Trade-off exists between speed, accuracy, and memory*
4. *Algorithm selection depends on application requirements*

Flat Index (Brute Force)

- *Compares all vectors*
- *High accuracy but very slow*

IVF (Inverted File Index)

1. *Clusters vectors and searches within relevant groups*
2. *Reduces computation significantly*
3. *Requires training before use*
4. *Faster but slightly less accurate*

HNSW

1. *Uses a multi-layer graph structure*
2. *Enables fast navigation across vectors*
3. *Provides high recall and speed*
4. *Requires higher memory*

PQ (Product Quantization)

1. *Compresses vectors to reduce memory usage*
2. *Trades accuracy for efficiency*
3. *Suitable for large-scale datasets*
4. *Improves scalability*

Assignment (API Setup)

1. *API key is securely obtained using getpass()*
2. *Stored in an environment variable (GROQ_API_KEY)*
3. *Used for authenticated API access*
4. *Model (llama-3.1-8b-instant) is explicitly defined for flexibility*

Routing and Orchestration

1. *Experts are defined in a configuration dictionary*
2. *Routing uses temperature = 0 for consistent classification*
3. *Only category names are returned to reduce ambiguity*
4. *Orchestrator selects appropriate system prompts dynamically*
5. *Different temperatures are used strategically for accuracy vs creativity*

System Performance

All queries are successfully routed to the appropriate experts.

- *Combines rule-based checks (for tools) and LLM-based classification*
- *Ensures consistent and accurate routing*
- *Orchestrator dynamically selects correct components*
- *Improves overall reliability and efficiency*

```
PS C:\Users\ABHISHEK\OneDrive\Documents\GenAI\GenAI-lab\GenAI-Hands-on> & C:/Users/ABHISHEK/AppData/Local/Programs/Python/Python312/python.exe "c:/Users/ABHISHEK/OneDrive/Documents/GenAI/GenAI-lab/GenAI-Hands-on/unit 2/moe_router.py"
Smart Customer Support Router (MoE) - type 'exit' to quit

You: My python script is throwing an IndexError on line 5.
Routed to: technical
Assistant: IndexError is typically caused by trying to access an element in a list or other sequence that doesn't exist. To determine the cause, I'll need more information about your script.

However, here's a general approach to troubleshoot the issue:

1. **Verify the list or sequence length**: Check if the list or sequence you're trying to access has elements. You can do this by printing its length or checking if it's empty.

2. **Check the indexing**: Make sure you're not trying to access an index that's out of bounds. In Python, list indices start at 0, so if your list has 5 elements, the last valid index is 4.

3. **Share your code**: If you're still unsure, please share the relevant part of your code, including the line throwing the IndexError and the surrounding code.

Here's a minimal example of how you might encounter an IndexError:

```python
my_list = [1, 2, 3, 4]
index = 5  # out of bounds
print(my_list[index])  # IndexError: list index out of range
```

To fix this, you could add a check to ensure the index is valid:

```python
my_list = [1, 2, 3, 4]
index = 5
if index < len(my_list):
    print(my_list[index])
else:
    print("Index out of bounds")
```

Without more information about your specific script, it's difficult to provide a more precise solution. If you're still experiencing issues, please provide more context or the exact error message.
```

```
PS C:\Users\ABHISHEK\OneDrive\Documents\GenAI\GenAI-lab\GenAI-Hands-on> & C:/Users/ABHISHEK/AppData/Local/Programs/Python/Python312/python.e
e "c:/Users/ABHISHEK/OneDrive/Documents/GenAI/GenAI-lab/GenAI-Hands-on/unit 2/moe_router.py"
Smart Customer Support Router (MoE) - type 'exit' to quit

You: I was charged twice for my subscription this month.
Routed to: billing
Assistant: I'm so sorry to hear that you've been charged twice for your subscription this month. This can be frustrating and unexpected.

To help resolve this issue, I'd like to review your account with you. Can you please provide me with the following information:

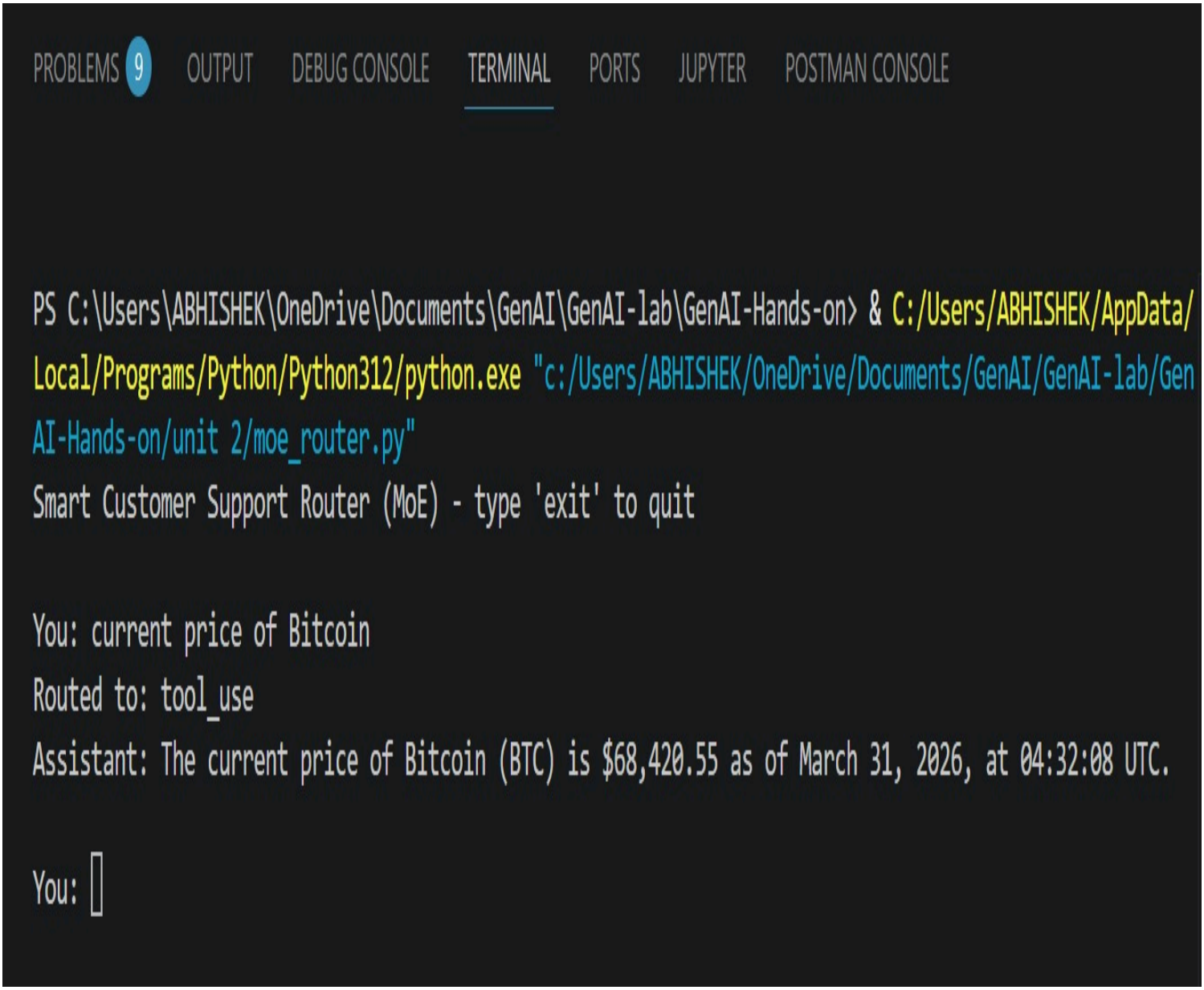
1. Your subscription details, including your account name and the type of subscription you have.
2. The dates on which the duplicate charges were made.
3. The amount of each charge.

Once I have this information, I'll be happy to look into this further for you. We have a clear policy in place to prevent duplicate charges,
and I'll work with you to resolve this issue as quickly as possible.

As per our billing policy, if a customer is charged twice for the same transaction, we will refund the duplicate charge immediately. Please
est assured that we take these types of issues seriously and will do our best to prevent them from happening in the future.

Is there anything else you'd like to add or discuss about your account?

You:
```


PROBLEMS 9
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
JUPYTER
POSTMAN CONSOLE
PS C:\Users\ABHISHEK\OneDrive\Documents\GenAI\GenAI-lab\GenAI-Hands-on> & C:/Users/ABHISHEK/AppData/Local/Programs/Python/Python312/python.exe "c:/Users/ABHISHEK/OneDrive/Documents/GenAI/GenAI-lab/GenAI-Hands-on/unit 2/moe_router.py"
Smart Customer Support Router (MoE) - type 'exit' to quit
You: current price of Bitcoin
Routed to: tool_use
Assistant: The current price of Bitcoin (BTC) is $68,420.55 as of March 31, 2026, at 04:32:08 UTC.
You: